Final Year Design Project

College Management System

for Govt. Graduate College, Mandi Bahauddin

By

Hadia
085668

Sharfa Kiran
085646

Syeda Laraib Qamar Kazmi
085713

Under the supervision of

Prof. Ubaid Ullah

Bachelor of Science in Information Technology (2022-2026)

DEPARTMENT OF INFORMATION TECHNOLOGY
GOVT. GRADUATE COLLEGE, MANDI BAHAUDDIN

College Management System for Govt. Graduate College, Mandi Bahauddin

A project presented to
Govt. Graduate College, Mandi Bahauddin

In partial fulfilment
of the requirement for the degree of

Bachelor of Science in Information Technology (2022-2026)

By

Hadia
085668

Sharfa Kiran
085646

Syeda Laraib Qamar Kazmi
085713

DEPARTMENT OF INFORMATION TECHNOLOGY
GOVT. GRADUATE COLLEGE, MANDI BAHAUDDIN

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted as part of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: _____
Hadia [085668]

Signature: _____
Sharfa Kiran [085646]

Signature: _____
Syeda Laraib Qamar Kazmi [085713]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSIT, session (2022-2026), titled "College Management System for Govt. Graduate College, Mandi Bahauddin" was developed by Hadia (085668), Sharfa Kiran (085646), and Syeda Laraib Qamar Kazmi (085713) under the supervision of Prof. Ubaid Ullah; in my opinion, it is fully adequate, in scope and quality, for the degree of Bachelor of Science in Information Technology.

Signature: _____
FYDP Advisor: Prof. Ubaid Ullah

Signatures (Faculty Advisory Committee – FAC)

	FAC1	**FAC2**
Name	Prof. Muhammad Faiyaz	Prof. Ubaid Ullah
Signature		

Signature: _____
Head of FYDP Coordination Office: Prof. Muhammad Faiyaz

Signature: _____ Dated: _____
Head of Department, Information Technology: Prof. Muhammad Faiyaz

Executive Summary

Govt. Graduate College, Mandi Bahauddin runs its academic life on paper. Roughly 3,000 students across Intermediate and BS programmes, and about 50 faculty members, still queue up at the accounts window for a fee challan, wait for a hand-written attendance register to be tallied, and find out their semester result from a notice board rather than a phone. Every one of these steps is a place where a form goes missing, a number gets mistyped, or a student simply doesn't find out in time. That was the starting problem: a college of this size cannot keep running its records by hand without the errors and delays showing up somewhere — a wrong CGPA on a transcript, a fee challan that doesn't match what the accounts office actually has on file, an exam schedule that reaches half the class.

Our answer is a suite of three Android applications — one for admins, one for teachers, one for students — backed by a single shared database and a common design system, so the same piece of data (a student's attendance, a session's fee structure, a published datesheet) is always the same regardless of which app is reading it. The admin app is where the college's actual structure lives: departments, the sessions (intakes) within them, each session's curriculum and timetable, and its fee structure. Teachers mark attendance, enter midterm and sessional marks, submit exam papers, and record semester GPA/CGPA against that same structure. Students see their own slice of it — attendance, marks, fee challan, datesheets, calendar notices — without ever walking into an office.

Midway through the project we made a deliberate architectural change: the backend moved from Firebase (Firestore + Auth) to Supabase (Postgres, with Row-Level Security enforcing who can see and write what, GoTrue for authentication, and Storage for uploaded files like exam papers and documents), and the UI layer moved from XML Activities/Fragments to Jetpack Compose with a proper MVVM/Repository split, wired together with Hilt. The reasoning was straightforward: a relational schema with real foreign keys and row-level policies gives us guarantees a document store couldn't — a mark can't be attributed to a student who was never enrolled in that session, a fee record can't outlive the session it belongs to — and Compose let us build one shared component library (the “Modernist” red/ink design system) that all three apps use identically instead of maintaining three separate sets of XML layouts.

Every major workflow in this report reflects that rebuilt system as it exists in the codebase today, not the original Firebase/Java prototype: department-centric academic structure, per-session fees, a lock-then-request-approval workflow for marks (a teacher enters a score once; changing it after the fact requires the admin's sign-off), semester result recording with GPA/CGPA and supply-subject tracking, fines, a calendar, datesheets, an uploadable-documents library, and a tiered Insights dashboard built directly on Postgres views so that what an admin sees college-wide, a teacher sees scoped to their own classes, purely through the database's own access rules — no extra application logic required. All three apps build and run against the live Supabase project.

Keywords: College Management System, Row-Level Security, Jetpack Compose, Supabase, Android

Acknowledgement

[PLACEHOLDER — write this yourself, in your own words. It's the one section of this report a template can't fill in for you. Typically thanks: Prof. Ubaid Ullah (your FYDP advisor) for supervision and guidance; the FYP Coordination Office / Prof. Muhammad Faiyaz; the Department of Information Technology, Govt. Graduate College, Mandi Bahauddin; and family or anyone else who supported you through the project.]

Signature: _____
Hadia [085668]

Signature: _____
Sharfa Kiran [085646]

Signature: _____
Syeda Laraib Qamar Kazmi [085713]

Abbreviations

Abbreviations and acronyms used throughout this report, ordered alphabetically.

Abbreviation	Description
API	Application Programming Interface
BS	Bachelor of Science
BSIT	Bachelor of Science in Information Technology
CGPA	Cumulative Grade Point Average
CMS	College Management System
CRUD	Create, Read, Update, Delete
DI	Dependency Injection (Hilt)
DTO	Data Transfer Object
FAC	Faculty Advisory Committee
FR	Functional Requirement
FYDP	Final Year Design Project
GGC MBD	Govt. Graduate College, Mandi Bahauddin
GPA	Grade Point Average
JWT	JSON Web Token
MVVM	Model-View-ViewModel
NFR	Non-Functional Requirement
RLS	Row-Level Security (Postgres)
RPC	Remote Procedure Call (a Postgres function invoked from the app)
SDK	Software Development Kit
SRS	Software Requirements Specification
UI	User Interface
UUID	Universally Unique Identifier

Chapter 1

Introduction

1. Introduction

This chapter lays out why the College Management System (CMS) exists, what it is meant to achieve, and how it is built. It covers the problem the college was actually living with, the three-app solution we built for it, the concrete objectives that solution targets, its scope, the modules that make it up, how it compares to systems like it, and the tools that went into building it.

1.1 Problem Statement

Govt. Graduate College, Mandi Bahauddin has roughly 3,000 students enrolled across its Intermediate and Bachelor's programmes, taught by about 50 faculty members, and every one of its academic and administrative processes is still run on paper. Fee challans are written out by hand at the accounts office, which means a student cannot get one without physically visiting during office hours and waiting in a queue. Attendance is marked in a physical register per class, then manually tallied at the end of the term to decide who meets the minimum-attendance requirement for exams — a process that is slow and open to arithmetic error. Midterm and sessional marks are recorded on paper sheets before being compiled into a semester result and GPA/CGPA by hand, with no independent check on that compilation beyond the teacher's and admin's own diligence. Exam datesheets and college notices are posted on physical noticeboards, so a student who does not visit campus on the right day may simply miss them.

None of this is a single catastrophic failure — it is the ordinary way many colleges of this size and resourcing operate. But it accumulates: a misread number on a marks sheet becomes a wrong CGPA on a transcript; a fee challan written from memory can disagree with what the accounts office has on record; an attendance shortfall discovered only at the end of the semester leaves no time to correct it. And every one of these processes requires the student or teacher to be physically present at the right office at the right time, which is the part that does not scale as enrolment grows. The problem this project addresses is exactly that: how does a college of this size keep its records accurate, timely, and reachable from outside the campus, without expanding its administrative staff to match?

1.2 Problem Solution

The solution is a suite of three role-based Android applications — cmsadmin, cmsteacher, and cmsstudent — that all read and write the same Supabase (Postgres) database, so a fact recorded once (a student's attendance for the day, a session's fee structure, a published exam datesheet) is visible identically to everyone who is allowed to see it, and to no one who isn't. Which app you sign into decides what you can do, not which data exists.

The admin app owns the college's structure: departments, the sessions (intakes) inside each department, each session's curriculum and weekly timetable, its fee structure, and its roster of students. From there, admin also runs the approval queues (student account linking, teacher-submitted mark-edit requests), the teacher lifecycle (disable/ban/reactivate, permission flags), and the notices side of the system — calendar events, datesheets, and an uploadable documents library — plus a tiered Insights dashboard for exam performance and at-risk students.

The teacher app is where the day-to-day academic work happens: marking attendance, entering midterm/sessional marks (locked after first entry, with a request-based path to correct a mistake), submitting exam papers, and recording semester GPA/CGPA including supply subjects. The student app is a read-mostly window onto all of it — attendance history, marks, semester results, fee challan, datesheets, calendar, and documents — plus the one write path a student has: submitting a link request to claim their own roll number.

Underneath all three, a shared library module (cmscommon) owns the domain models, the Supabase-backed repositories, an offline Room cache with a background SyncEngine for data that's read often (rosters, timetables, attendance), and the “Modernist” Compose design system so the three apps look and behave as one product rather than three separately-built ones. Postgres Row-Level Security — not application code — is what actually enforces who can read or write which row, which is what lets the same Insights queries return a college-wide view for admin and an automatically-scoped view for a teacher without any role-branching in the app.

1.3 Objectives of the Proposed System

The system is built around the following measurable objectives:

- Let a student generate and view their own fee challan from their phone, with zero in-person visits to the accounts office required for a routine, unmodified challan.

- Let a teacher mark a full class's daily attendance in under one minute per class, replacing the paper register with a digital one that is immediately visible to admin and the affected students.
- Enforce that a midterm or sessional score, once entered, cannot be silently changed — any correction must go through an explicit admin-approved request, closing the main source of unaudited mark changes.
- Give every student visibility into their own semester GPA/CGPA progression, including supply subjects, without waiting for a printed transcript.
- Publish exam datesheets and college notices (holidays, deadlines, events) to every affected student and teacher the moment they're published, replacing the noticeboard.
- Scope every piece of data — attendance, marks, fees, analytics — to the correct role automatically via database-level access rules (Postgres Row-Level Security), rather than relying on the application to remember to check permissions.
- Keep the system usable when connectivity drops, by caching the data a user reads most often (roster, timetable, attendance) locally and reconciling once the connection returns.

1.4 Scope

In scope: three native Android applications (admin, teacher, student) covering the full academic-and-administrative cycle for one college — departments, sessions, curriculum, timetable, attendance, marks, semester results, per-session fees, fines, calendar, datesheets, an uploadable-documents library, student account linking, teacher lifecycle management, and role-scoped analytics. The initial deployment target is Govt. Graduate College, Mandi Bahauddin; the department-centric data model (a department owns many sessions/intakes, each session owns its own curriculum, timetable, and fee structure) is generic enough to extend to other PU-affiliated colleges without a schema change, though that rollout is not part of this project.

Out of scope: web or iOS clients (Android only), online/electronic fee payment (the system generates and displays a challan; payment still happens at the accounts office), automated exam-paper grading, and a public-facing website. Stakeholders are the college administration (department heads, the accounts office, the FYP-equivalent college management), teaching faculty, and enrolled students — all three are directly represented by one of the three apps.

Priorities, in the order features were actually built: (1) identity and the department-centric academic structure — without departments/sessions/curriculum nothing else has anywhere to attach; (2) the day-to-day teacher workflows — attendance and marks, since these are the highest-frequency actions in the system; (3) the financial and notices layer — fees, fines, calendar, datesheets, documents; (4) governance and oversight — link-request approval, teacher lifecycle, and tiered analytics, since these depend on the first three layers already existing.

1.5 System Components

The system is organised as three client apps sharing one backend and one common library module. Each app's modules are listed below, grouped app by app.

1.5.1 Client Admin App (cmsadmin)

- Departments — create/list departments; each owns many sessions.
- Sessions — create/delete a session (intake) under a department; promote its current semester.
- Curriculum — per-session, per-semester subject list.
- Master Timetable & Session Timetable — weekly slot grid (subject/teacher/room) with double-booking prevention.
- Session Fee Structure — per-session fee heads, cadence, due date, payment note.
- Teachers — roster, account creation, permission flags, status lifecycle (disable/ban/reactivate).
- Link Requests — approve/reject a student's claim to a roll number.
- Attendance Records — dept → session → semester report + PDF/CSV export.
- Calendar — create/delete college-wide events (holidays, exams, deadlines) with audience targeting.
- Datesheets — build an exam grid (slots) and publish it.
- Documents — upload a PDF/DOCX or type body text (prospectus/rules/report/other) and publish it.

- Mark Edit Requests — review queue to approve/reject a teacher's request to change an already-locked score.
- Insights — college-wide session overview, at-risk students, exam-score statistics.
- Notifications — send/manage notices to teachers and/or students.

1.5.2 Client Teacher App (cmsteacher)

- Home — at-a-glance dashboard for the signed-in teacher.
- Mark Attendance + History — daily attendance entry per class, with a history view.
- Exams hub — Marks Entry (midterm/sessional, live negative/overflow validation, locks after first save), Submit Exam Paper, Semester Results (GPA/CGPA/supply), Datesheets (view published).
- Schedule — the teacher's own weekly timetable.
- Menu hub — My Students, Calendar, Documents, Insights (scoped to the teacher's own sessions/courses via RLS), Link Requests (for permitted teachers), Notifications, Profile.

1.5.3 Client Student App (cmsstudent)

- Home — attendance/CGPA snapshot and quick links.
- Attendance — the student's own attendance history.
- Exams hub — Marks (subject scores), Results (GPA/CGPA progression with supply), Datesheets (published only).
- Timetable — the student's own weekly schedule.
- More hub — Calendar, Documents, Fee Challan, Notifications, Profile.
- Link Request — a not-yet-linked account claims its roll number, name, CNIC/B-Form, and DOB for admin review.

1.5.4 Shared Library Module (cmscommon)

- Domain models, DTOs, and Repository interfaces/implementations — the single source of truth for every table each app touches.
- Supabase client wiring — Postgrest (database), Auth/GoTrue (sign-in), Storage (uploaded files: exam papers, documents), Edge Functions (admin-privileged actions like creating a teacher account or changing its status).
- Room offline cache + SyncEngine — background sync for high-read-volume data (roster, timetable, attendance); low-volume features (fees, fines, calendar, datesheets, documents, insights) read directly from Postgrest instead.
- The “Modernist” Compose design system — shared theme, typography, and components (LedgerCard, HubNavCard, SegmentToggle, StatusBadge, etc.) used identically by all three apps.

1.6 Related System Analysis / Literature Review

A handful of existing systems cover pieces of the same ground. None of them fit a Pakistani public-sector college's actual structure (departments → sessions/intakes → semesters, per-session fees, an accounts-office-based fee-collection process) out of the box.

Table 1-1 Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Google Classroom	Built around individual courses, not a college's administrative structure — no fee/challan concept, no institution-wide attendance percentage tracking, no per-session curriculum/timetable model.	Models the college itself (departments, sessions, curriculum, fees) as first-class data, not just course content.
Generic ERP-style college portals (common in Pakistani colleges)	Usually web-only, built for desktop use at an admin's desk; require a developer to touch the database directly for anything not in the original scope; marks corrections have no audit trail.	Native mobile apps for all three roles; every mark change after first entry goes through an explicit, logged approval request instead of a silent database edit.
Paper-based systems (the	No real-time visibility for	Every role sees the same live data

status quo at GGC MBD)	students/teachers, no cross-checking of manually compiled GPA/CGPA, notices depend on physically visiting a noticeboard.	instantly; RLS enforces who can see/write what without a physical visit.

1.7 Vision Statement

For the administration, faculty, and students of Govt. Graduate College, Mandi Bahauddin,

Who need accurate, timely, and remotely-reachable academic and administrative records instead of paper-based ones,

The College Management System (CMS)

Is a suite of three role-based Android applications backed by a single shared database,

That lets a student check attendance, marks, results, and fees from their phone, a teacher mark attendance and enter results digitally with a built-in correction workflow, and an admin manage the college's entire academic structure and oversee it through a role-scoped analytics dashboard,

Unlike the current paper-based process, which requires an in-person visit for almost every interaction and has no audit trail for a changed mark,

Our product makes every one of those interactions instant, auditable, and reachable from outside the campus, while still working offline for the data a user reads most.

1.8 System Limitations and Constraints

Limitations (external, beyond our control):

- Requires an internet connection for any write and for data not already cached locally; the offline cache only covers high-read-volume data (roster, timetable, attendance).
- Android only — no iOS or web client exists.
- Depends on Supabase's free-tier limits (storage quota, function invocations) at the current scale of deployment.

Constraints (self-imposed, within our control):

- Fee collection stays a two-step process by design — the app generates and displays the challan, but payment is still made at the college accounts office, not through an in-app payment gateway.
- A score is locked after its first entry; any change afterward must go through the mark-edit-request workflow rather than a direct edit, even for the teacher who entered it.
- The initial deployment targets a single college (GGC MBD); multi-college support is architecturally possible (departments are already scoped) but not exercised or tested.

1.9 Tools and Technologies

Versions below are pinned in the project's Gradle version catalog (gradle/libs.versions.toml), so they reflect what the codebase actually builds against, not an aspirational list.

Tool / Technology	Version	Rationale
Kotlin	2.1.0	Primary language for all three apps and the shared module.
Jetpack Compose (BOM)	2024.12.01	Declarative UI toolkit; lets all three apps share one component library instead of duplicating XML layouts.
Hilt	2.52	Dependency injection across ViewModels/Repositories.
Room	2.6.1	Local offline cache for high-read-volume data (roster, timetable, attendance) with a background SyncEngine.
Supabase Kotlin SDK (BOM)	3.1.4	Postgrest (database), Auth/GoTrue, Storage,

		Functions, Realtime — the entire backend client.
Ktor	3.1.1	HTTP engine the Supabase SDK runs on.
kotlinx.serialization	1.8.0	JSON (de)serialization for every DTO exchanged with Postgrest.
Backend: Postgres + Row-Level Security	Supabase-managed	Enforces per-role read/write access at the database layer instead of in application code.
Android Studio	current stable	IDE for all development.
Git	—	Version control across the monorepo (3 apps + shared module + Supabase migrations).

1.10 Project Deliverables

- This FYDP report, covering requirements, design, implementation, testing, and deployment.
- Three installable Android applications: cmsadmin, cmsteacher, cmsstudent (debug/release APKs).
- The Supabase backend: schema migrations, RLS policies, Edge Functions, and storage bucket configuration (in supabase/).
- Source code for all three apps and the shared cmscommon module, under version control.
- This report's data dictionary and functional-requirements tables, which double as the closest thing to a standalone SRS for this project.

1.11 Project Planning

[NOTE — reconstructed, not the original academic plan: the timeline below is built from actual git commit history (Jan 14 – Jun 17 2026) for the original Java/Firebase build, plus session records for the Kotlin/Compose/Supabase rebuild (Jul 2026), since that later work was never committed to git as individual dated commits. Please confirm the real FYDP milestone/deadline dates and swap them in if they differ.]

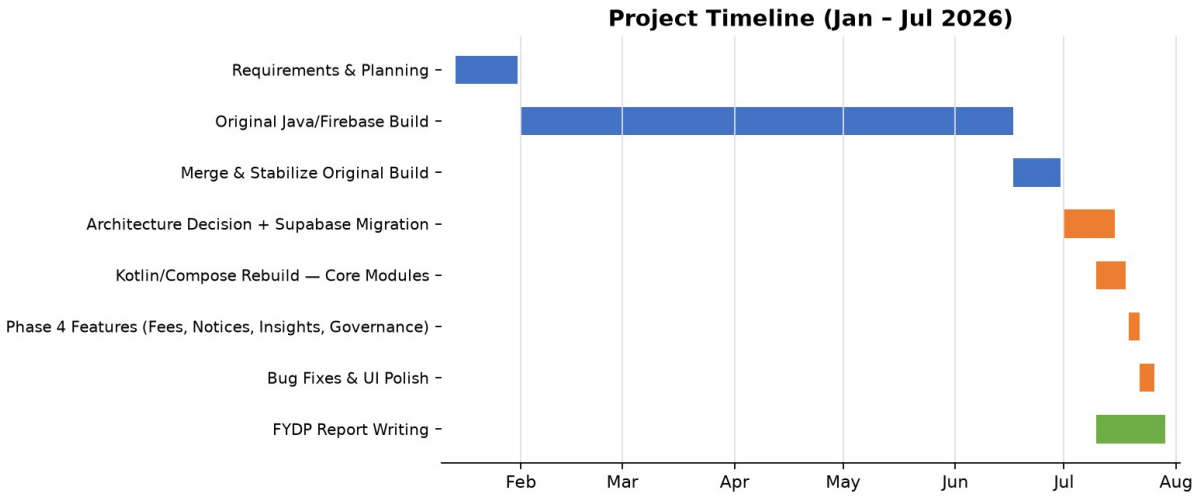

Figure 1-1 Project timeline, reconstructed from commit history and session records

1.12 Summary

This chapter set out the actual problem — a college running entirely on paper records — and the three-app, Supabase-backed solution built to address it, along with the concrete objectives, scope, module breakdown, and technology choices behind that solution. The remaining chapters go into that solution in depth: Chapter 2 specifies its requirements formally, Chapter 3 covers its design and data model, Chapter 4 its implementation, Chapter 5 its testing, Chapter 6 its deployment, and Chapter 7 closes by checking the finished system against the objectives set out here.

Chapter 2

Requirements Analysis

2. Analysis

Chapter 1 described what the system is and why it exists. This chapter pins that down into requirements a reviewer could actually check the finished apps against: who uses the system, how those requirements were gathered, and — in the sections that follow — the specific functional and non-functional requirements themselves.

2.1 User Classes and Characteristics

Three roles use the system, and a fourth distinction — permission flags on the teacher role — matters enough to call out separately.

User Class	Characteristics
Admin	College staff who run the academic structure end to end: departments, sessions, curriculum, timetable, fees, teacher accounts, and the approval queues (student link requests, mark-edit requests). Comfortable with a college's actual bureaucracy — the roles map closely to what an accounts office or registrar already does on paper. Full read/write access, gated by nothing but their own admin flag in Postgres.
Teacher (base)	Faculty who mark attendance, enter marks for the classes they teach, submit exam papers, and record semester results. Their write access is scoped by the college timetable itself — a teacher can only touch a session/course they're actually assigned to teach, enforced at the database level rather than trusted to the app.
Teacher (with permission flags)	Same as above, plus one or more of: approving student link requests, editing the master timetable, sending notifications, or managing datesheets — each a boolean flag an admin sets on that teacher's account, not a separate role.
Student	Enrolled students checking their own attendance, marks, semester results, fee challan, datesheets, and college notices. Read-only everywhere except their own link-request submission — RLS scopes every query to session_id = my_session() so a student physically cannot query another student's row even by guessing an ID.

2.2 Requirement Identifying Technique

Use-case modeling was the main technique. Every one of the three apps is an interactive, session-based tool where a specific actor triggers a specific action and expects a specific result — mark this student present, publish this datesheet, approve this fee change — which is exactly the shape a use case captures well. Storyboarding wasn't a fit here: nothing in the system is graphically driven enough to need it, and the UI itself (the shared "Modernist" Compose component set) was designed after the use cases and requirements, not before them.

In practice, requirements came from watching how the college's existing paper process actually worked — who fills in what, who signs off on it, what happens when someone makes a mistake — and then asking what a digital equivalent of that step would need to do. The mark-lock-then-request-edit requirement is a direct example: it exists because the paper process already had an informal version of it (a teacher didn't get to unilaterally change a mark sheet once it was submitted to the office), so the requirement formalizes an existing norm rather than inventing a new one.

2.3 Functional Requirements

The requirements below are grouped by the module that owns them — academic structure, attendance/marks, financial, notices, and governance/analytics — which follows how the admin app itself is organized rather than an arbitrary alphabetical list. Each one uses the same eight-attribute template so a reviewer can cross-reference it against the traceability matrix in Chapter 7 without guessing at what "source" or "business rule" means from row to row.

2.3.1 FR-1: Manage Departments

Identifier	FR-1
Title	Manage Departments

Requirement	The Admin shall be able to create and list departments. Each department is a top-level container that owns its own sessions (intakes); nothing else in the system can exist without a department to attach to.
Source	College administration — departments are the college's real organisational unit (e.g. Computer Science, Information Technology).
Rationale	Every other piece of academic data (sessions, curriculum, fees, students) is scoped under a department, so this is the first thing that has to exist.
Business Rule	A department's ID cannot contain underscores, since session IDs are built as {deptId}_{startYear}_{shift} and are parsed back apart by splitting on underscore.
Dependencies	None — this is the root of the data model.
Priority	High

2.3.2 FR-2: Manage Sessions

Identifier	FR-2
Title	Manage Sessions
Requirement	The Admin shall be able to create a session (an intake, e.g. "2021 Morning") under a department, delete it, and promote its current semester as the class progresses. Deleting a session shall cascade to its students, timetable, and curriculum.
Source	College administration — a session is a specific batch of students admitted in a given year and shift.
Rationale	Attendance, marks, fees, and timetable all key off a session, not just a department, because two intakes of the same program can be at different semesters simultaneously.
Business Rule	Session ID is built as {deptId}_{startYear}_{shift}; shift is MORNING or EVENING.
Dependencies	FR-1 (a session cannot exist without its department).
Priority	High

2.3.3 FR-3: Manage Curriculum

Identifier	FR-3
Title	Manage Curriculum
Requirement	The Admin shall be able to set the subject list for a given session and semester (course code, name, credit hours), and replace that list wholesale when it needs correcting.
Source	College administration, following the program's official scheme of studies.
Rationale	Curriculum is stored per-session rather than per-department because two intakes of the same program can legitimately differ — a dropped elective, a renumbered course — and the timetable/marks/attendance screens all need to know which subjects apply to which session's current semester.
Business Rule	Saving a semester's subject list deletes the old rows for that session+semester and inserts the new ones, rather than diffing and patching — simpler to reason about, and curriculum changes are infrequent enough that this isn't a performance concern.
Dependencies	FR-2 (a session must exist first).
Priority	High

2.3.4 FR-4: Manage Session Timetable

Identifier	FR-4
Title	Manage Session Timetable
Requirement	The Admin shall be able to assign a subject, teacher, and room to a weekly time slot for a session, and the system shall refuse to save a slot that would double-book a teacher or a room at the same day and time.
Source	College administration and the master timetable process.
Rationale	A teacher physically cannot be in two rooms at once; catching that at save-time is cheaper than discovering it on the first day of class.
Business Rule	Double-booking is checked with a database trigger, not app-side logic, since the admin app and any future client writing to the same table would otherwise need to duplicate the check.
Dependencies	FR-2, FR-3 (a slot references a session's existing subjects).
Priority	Medium

2.3.5 FR-5: Mark Attendance

Identifier	FR-5
Title	Mark Attendance
Requirement	A Teacher shall be able to mark a class's attendance for the current day in a single screen, and view the attendance history for a session/course they teach.
Source	Daily classroom practice — replaces the paper register.
Rationale	This is the single most frequent write in the whole system (once per class, per day, per teacher), so it needs to be fast — the whole class in one screen, not one row at a time.
Business Rule	A teacher can only mark attendance for a session/course they're actually assigned to on the timetable; enforced by RLS (teaches(session_id)), not just hidden in the UI.
Dependencies	FR-4 (a teacher's assignment comes from the timetable).
Priority	High

2.3.6 FR-6: Enter Marks (Locks on First Entry)

Identifier	FR-6
Title	Enter Marks
Requirement	A Teacher shall be able to enter midterm (out of 25) or sessional (out of 15) scores for every student in a class they teach. The field shall reject a negative value or a value above the exam's maximum as the teacher types, before they even try to save. Once a score is saved for a student, it becomes read-only in this screen — a further change has to go through FR-7.
Source	Direct requirement — see 1.3 objectives, and the current problem of unaudited mark changes.
Rationale	Catching an out-of-range score live (rather than after Save, or worse, not at all) prevents a mistyped '250' from ever reaching the database in the first place.
Business Rule	The Save action only writes rows for students who don't already have a saved score — it never re-submits an existing row, since the database itself would reject that update (see FR-7's business rule).

Dependencies	FR-4 (a teacher's class assignment), FR-3 (which exam types/max-marks apply).
Priority	High

2.3.7 FR-7: Request Mark Edit

Identifier	FR-7
Title	Request Mark Edit
Requirement	For a student whose score is already saved (locked), a Teacher shall be able to submit a request with the proposed new score and an optional reason, instead of editing the score directly.
Source	Direct requirement from the college — a mark, once submitted, shouldn't be silently changeable by the same person who entered it.
Rationale	This is the audit trail the paper process never had: every correction after the fact is now a named, timestamped record instead of an invisible edit.
Business Rule	The database enforces this, not just the UI — the session_marks UPDATE policy is admin-only, so even a teacher who bypassed the app couldn't directly overwrite a saved score.
Dependencies	FR-6 (there has to be a saved score to request a change to).
Priority	Medium

2.3.8 FR-8: Approve/Reject Mark Edit Request

Identifier	FR-8
Title	Approve/Reject Mark Edit Request
Requirement	The Admin shall see every pending mark-edit request in one queue, with the current score, the requested score, and the teacher's reason, and shall be able to approve or reject each one. Approving writes the new score onto the student's record and closes the request; rejecting just closes it.
Source	Direct requirement — the other half of FR-7.
Rationale	Centralising this in one queue means an admin doesn't have to go hunting through individual classes to find what's pending.
Business Rule	Approval is two sequential writes (update the score, then mark the request approved), not a single atomic transaction — acceptable here because both writes are admin-gated and idempotent if one had to be retried.
Dependencies	FR-7.
Priority	Medium

2.3.9 FR-9: Record Semester Result

Identifier	FR-9
Title	Record Semester Result
Requirement	A Teacher shall be able to record a student's GPA, CGPA, result status (promoted/repeated/probation/pending), class position, remarks, and any supply (retake) subjects for a completed semester.
Source	End-of-semester result compilation, currently done by hand.
Rationale	This is where the paper process's biggest error risk lived — a hand-compiled CGPA with no independent check. Recording it through one RPC call means the CGPA

	snapshot on the student's record updates atomically with the semester row.
Business Rule	The snapshot only updates when the semester being recorded is the highest one seen so far for that student — so if an 8th-semester student's result gets entered before catching up an earlier gap, the snapshot doesn't jump ahead incorrectly.
Dependencies	FR-2, FR-6 (marks feed into the GPA calculation, though the calculation itself happens outside the app).
Priority	High

2.3.10 FR-10: Manage Session Fee Structure

Identifier	FR-10
Title	Manage Session Fee Structure
Requirement	The Admin shall be able to set a session's fee cadence (semester or annual), its fee heads (label + amount, e.g. Tuition, Lab), due date, late-fine note, and payment note.
Source	College accounts office fee schedule.
Rationale	Fees were originally modelled per-department; a project-mid-course correction moved them to per-session, since two intakes of the same department can have different fee schedules (a new intake's tuition can rise year over year).
Business Rule	Saving fee heads deletes-then-inserts the whole set for that session, same pattern as curriculum (FR-3) — a small, infrequent write where correctness matters more than avoiding a full rewrite.
Dependencies	FR-2.
Priority	High

2.3.11 FR-11: View Fee Challan

Identifier	FR-11
Title	View Fee Challan
Requirement	A Student shall be able to view their own session's fee structure — cadence, fee heads, total, due date, and payment note — without visiting the accounts office.
Source	Direct requirement — this is the objective from Chapter 1 about generating a fee challan without an office visit.
Rationale	This is informational only; it doesn't process a payment. That was a deliberate scope call (see 1.4/1.8) — the accounts office still collects the money, this screen just removes the reason to visit in person beforehand.
Business Rule	None beyond RLS scoping the query to the student's own session.
Dependencies	FR-10.
Priority	High

2.3.12 FR-12: Issue Fine

Identifier	FR-12
Title	Issue Fine
Requirement	The Admin shall be able to issue a fine against a student's record with a category (library, attendance, exam, disciplinary, other), amount, and reason.
Source	College disciplinary/library process.

Rationale	Fines needed to attach to the student's existing profile screen rather than live as a separate module, since an admin issuing one is usually already looking at that student's record for another reason.
Business Rule	None beyond admin-only write access.
Dependencies	FR-2 (a student has to exist in a session first).
Priority	Low

2.3.13 FR-13: View Fines

Identifier	FR-13
Title	View Fines
Requirement	A Student shall be able to see a read-only list of fines issued against them, with a running total.
Source	Direct requirement — pairs with FR-12.
Rationale	A student finding out about a fine through the app instead of at graduation clearance is the whole point of digitising this.
Business Rule	None.
Dependencies	FR-12.
Priority	Low

2.3.14 FR-14: Manage Calendar Events

Identifier	FR-14
Title	Manage Calendar Events
Requirement	The Admin (or a permitted Teacher) shall be able to create and delete college-wide calendar events — holidays, exam dates, deadlines — and target them to students, teachers, or everyone.
Source	Replaces the physical noticeboard for date-based announcements.
Rationale	Audience targeting exists because a deadline for teachers (grade submission) and a deadline for students (fee due date) shouldn't both show up cluttering everyone's calendar.
Business Rule	Write access requires either is_admin() or a teacher with can_manage_datesheets-equivalent permission; read access is open to all authenticated roles.
Dependencies	None.
Priority	Medium

2.3.15 FR-15: Build & Publish Datesheet

Identifier	FR-15
Title	Build & Publish Datesheet
Requirement	The Admin shall be able to create a datesheet (title, exam type, instructions), add exam slots to it (date, time, course, room, invigilator), and toggle it between draft and published.
Source	Exam scheduling process — replaces the printed datesheet pinned to the noticeboard.
Rationale	A draft/published split matters because a datesheet is rarely finished in one sitting —

	an admin needs to build it over several sessions before it's ready for students to see.
Business Rule	Students and non-managing teachers can only read published datesheets (see FR-16); drafts are only visible to admin and teachers with datesheet-management permission.
Dependencies	FR-2, FR-3.
Priority	Medium

2.3.16 FR-16: View Datesheet

Identifier	FR-16
Title	View Datesheet
Requirement	A Teacher or Student shall be able to browse published datesheets and expand one to see its exam slots.
Source	Direct requirement — the read side of FR-15.
Rationale	None beyond the obvious — a student needs to know when their own exams are.
Business Rule	Enforced by RLS: only rows with published = true are visible to a non-managing role.
Dependencies	FR-15.
Priority	Medium

2.3.17 FR-17: Upload & Publish Document

Identifier	FR-17
Title	Upload & Publish Document
Requirement	The Admin shall be able to either upload a PDF/DOCX file or type body text directly, tag it with a kind (prospectus, rules, report, other) and an audience, and publish it.
Source	College documents that currently only exist as physical printouts (prospectus, rulebook).
Rationale	Giving the admin a choice between uploading a file and just typing text covers both cases — a rulebook that's genuinely a formatted PDF, and a one-off short notice that doesn't deserve a whole file.
Business Rule	Uploaded files live in a private Supabase Storage bucket; a document under the archives/ path is intentionally excluded from the public-read policy, reserved for internal-only material.
Dependencies	None.
Priority	Low

2.3.18 FR-18: View/Download Document

Identifier	FR-18
Title	View/Download Document
Requirement	A Teacher or Student shall be able to browse published documents, read inline body text, and download an attached file to open it locally.
Source	Direct requirement — the read side of FR-17.
Rationale	None beyond the obvious.
Business Rule	Downloaded files are opened through the device's own file-provider/viewer rather than an in-app viewer, since supporting every document format in-house wasn't worth

	building.
Dependencies	FR-17.
Priority	Low

2.3.19 FR-19: Submit Student Link Request

Identifier	FR-19
Title	Submit Student Link Request
Requirement	A newly-registered account with no linked student record shall be able to submit a claim — session, roll number, name, CNIC/B-Form, date of birth — asking to be linked to that roll number.
Source	Onboarding — a student's Supabase auth account and their session_students roster row don't exist as the same thing until this link happens.
Rationale	An unlinked account can't read the roster to check whether its claimed roll number is even real, so the claim itself has to be a blind submission that admin verifies server-side.
Business Rule	The (session, roll) pair is verified against the actual roster only at approval time (FR-20), not at submission time, since RLS prevents an unlinked account from querying the roster directly.
Dependencies	FR-2 (the session and roll have to already exist in the roster for the claim to succeed).
Priority	Medium

2.3.20 FR-20: Approve/Reject Link Request

Identifier	FR-20
Title	Approve/Reject Link Request
Requirement	The Admin (or a permitted Teacher) shall see every pending link request, verify the claimed roll number actually exists in that session's roster, and approve or reject it. Approving revokes any account previously linked to that same roll number.
Source	Direct requirement — the other half of FR-19.
Rationale	Revoking a prior link on approval closes an obvious abuse case: without it, a roll number could end up claimed by two different accounts simultaneously.
Business Rule	The match check (does this session+roll exist?) happens server-side at approval time, using the reviewer's own admin/teacher privileges — the requester's account never gets roster read-access itself.
Dependencies	FR-19.
Priority	Medium

2.3.21 FR-21: Manage Teacher Status & Permissions

Identifier	FR-21
Title	Manage Teacher Status & Permissions
Requirement	The Admin shall be able to disable, ban, or reactivate a teacher's account, and toggle their permission flags (approve link requests, edit timetable, send notifications, manage datesheets).
Source	College HR/administrative process for faculty account lifecycle.

Rationale	Disabling or banning immediately revokes that teacher's ability to sign in — it bans the underlying auth account, not just a flag the app checks — so access actually stops the moment it's toggled, not the next time some check happens to run.
Business Rule	A disabled or banned teacher still appears in the roster (so admin can reactivate them later); only an outright delete removes the row.
Dependencies	None.
Priority	Medium

2.3.22 FR-22: View Tiered Insights

Identifier	FR-22
Title	View Tiered Insights
Requirement	The Admin shall see college-wide session overviews, at-risk students (low attendance or CGPA), and exam-score statistics. A Teacher shall see the same three views scoped automatically to only the sessions and courses they teach.
Source	Direct requirement — analytics for oversight, one of the last features built.
Rationale	"Tiered" doesn't mean two separate screens with two separate query paths — it's the same three Postgres views (session_overview, at_risk_students, exam_stats) queried by both roles. Row-Level Security on the underlying tables does the scoping, so there's no role-branching logic in the app at all.
Business Rule	session_overview joins from a world-readable table (academic_sessions), so a teacher's query technically returns a row for every session college-wide — just with student counts and averages blank for sessions they don't teach, since RLS blocks the joined rows, not the session row itself. No student-level data ever leaks through this.
Dependencies	FR-5, FR-6, FR-9 (the underlying views read attendance, marks, and GPA data).
Priority	Low

2.4 Non-Functional Requirements

2.4.1 Reliability

High-read-volume data — roster, timetable, attendance — is cached locally in Room and kept current by a background SyncEngine, so a teacher marking attendance mid-class doesn't lose the roster if the connection drops for a few seconds. Lower-volume features (fees, fines, calendar, datesheets, documents, insights) read directly from Postgrest instead of maintaining a local copy, since they're checked far less often and a brief loading spinner on a weak connection is an acceptable cost for not maintaining a second cache layer.

Failure mode when a write genuinely can't reach the server: the screen shows the error instead of silently discarding the action — there's no queued-retry mechanism for writes yet, which is a real gap the Future Work section (7.4) calls out rather than papers over.

2.4.2 Usability

All three apps share one Compose component library, so a Ledger card, a section header, or a segmented toggle looks and behaves identically whether it's in the admin, teacher, or student app — someone who's learned one app already knows the visual language of the other two. Marks entry validates as the teacher types rather than only on submit, and every destructive action (deleting a session, banning a teacher) goes through a confirmation dialog that states what else gets affected, not just a bare "Are you sure?".

One real usability bug did slip through and got caught late: a shared segmented-toggle component had an unconstrained divider that, under certain screen heights, expanded to swallow the rest of the screen below the marks-entry exam-type selector. It was found from a user screenshot, not a design review, which says something about the limits of not having done any structured usability testing yet — noted honestly in Chapter 6's Challenges section rather than left out.

2.4.3 Performance

Screens backed by the Room cache (roster, timetable, attendance) render from local data immediately and sync in the background, so there's no network round-trip on the critical path for the highest-frequency actions. Screens that read directly from Postgrest (fees, insights, documents) accept a network round-trip per load since they're checked far less often — the tradeoff is simplicity over shaving a few hundred milliseconds off an infrequent screen.

No load testing has been run against the Supabase project at anything beyond the current single-college scale; the free-tier connection and storage limits are the practical ceiling right now, and that's an open item rather than a validated number.

2.4.4 Security

Every table is protected by Postgres Row-Level Security, so access control lives in the database, not in application code that a bug could accidentally skip. A student's queries are scoped to session_id = my_session() and roll_number = my_roll(); a teacher's are scoped through a teaches(session_id) function keyed off the timetable; admin bypasses both via an is_admin() check. Authentication itself is Supabase's GoTrue (JWT-based), so the app never handles or stores a raw password.

The clearest proof this actually holds is the Insights feature (FR-22): the exact same SQL query, run by an admin, a teacher, or a student, returns a different result set for each — not because the app checked their role and branched, but because RLS decided what rows existed for that JWT before the app ever saw them.

2.5 External Interface Requirements

2.5.1 User Interfaces Requirements

All three apps follow one shared design system internally called "Modernist" — a flat red/ink Archivo-based look with square (non-rounded) cards, a consistent eyebrow/title/rule header pattern, and one shared set of components (LedgerCard, HubNavCard, SegmentToggle, StatusBadge, and similar). Bottom navigation follows Android's standard 5-tab pattern; destructive actions always show a confirmation dialog naming what else is affected, not a bare confirm/cancel. There's no separate accessibility pass yet — standard Material3 contrast and touch-target defaults apply, nothing beyond that has been verified.

2.5.2 Software Interfaces

Supabase (self-hosted-compatible, currently cloud-hosted) provides four services the app talks to directly: Postgrest for the database, GoTrue for authentication, Storage for uploaded files, and Edge Functions for the handful of actions that need service-role privileges (creating a teacher account, changing a teacher's status) rather than what the signed-in user's own RLS permissions allow. Locally, Room provides the offline cache described in 2.4.1. All three apps and the shared cmscommon module talk to Supabase through the official supabase-kt SDK over Ktor.

2.5.3 Hardware Interfaces

No special hardware — a standard Android phone or tablet is all any of the three apps need. Nothing uses the camera, biometric sensors, or any other device peripheral; a document upload picks an existing file from the device's storage rather than scanning one.

2.5.4 Communications Interfaces

All network traffic is HTTPS/REST against the Supabase project, via Ktor. Account verification and password-reset flows go through GoTrue's built-in email delivery, which means the college's SMTP configuration on the Supabase dashboard is a real dependency for those two flows working at all — noted as a standing deployment item in Chapter 6.

2.6 Summary

This chapter turned Chapter 1's goals into something checkable: three user classes, use-case-driven requirement gathering rooted in the college's actual paper process, twenty-two functional requirements spanning academic structure through governance and analytics, and the non-functional and interface requirements that constrain how all of it has to behave. Chapter 3 now covers how these requirements were turned into an actual system design — the data model, the architecture, and the decisions made along the way.

Chapter 3

Design and Architecture

3. System Design

Chapter 2 fixed what the system has to do. This chapter is about how it actually does it: the structural decisions, the data model, and the trade-offs that came with each one — including the ones that didn't work on the first attempt.

3.1 Design Considerations

Assumptions and Dependencies

The whole design assumes Supabase's security_invoker views behave correctly — a view marked security_invoker re-runs the underlying tables' RLS policies for whoever queries it, rather than running with the view owner's privileges. Tiered access (FR-22) depends entirely on that one Postgres feature working as documented; if it didn't, the Insights feature would need per-role query branching instead of one shared view.

Limitations

Room's schema is versioned but not migrated — every version bump so far has used fallbackToDestructiveMigration, which wipes the local cache rather than writing a real migration path. That's a deliberate pre-launch shortcut: with no users yet, there's nothing to lose by wiping the cache, and it's saved real time across nine schema bumps. It stops being acceptable the moment the app is actually deployed with live user data on a device, since a schema change would then delete a real user's offline cache.

Risks

The biggest risk found during the build wasn't a design flaw so much as an interaction between two correct-looking pieces: kotlinx.serialization drops any DTO field that equals its Kotlin default (encodeDefaults=false), and Postgrest's bulk upsert fills in NULL for any column missing from a row in a multi-row insert. Combine the two and a bulk write where most rows share a field's default value can silently NULL that column out for every row — not just the ones where the value was genuinely absent. It surfaced three separate times before the pattern was recognised (see 3.7), and the fix is now a standing rule: any DTO field whose column is NOT NULL with no database default gets a sentinel default that can never occur for real (empty string, not the semantically-meaningful default).

3.2 Design Models

Three kinds of diagram cover this system, not all of the object-oriented standard set. An architectural component diagram (3.3) shows how the layers fit together. A data dictionary (3.4) stands in for a class diagram — the domain here is genuinely closer to a relational schema than an object graph, since almost every domain "object" is a thin Kotlin data class mapped straight onto a Postgres row, with the real behaviour living in Repository functions and RLS policies rather than in methods on the objects themselves. Sequence diagrams (3.6) cover the four flows worth walking through step by step: login, the marks lock/edit-request/approval loop, attendance marking, and document publish/download.

A state-transition diagram was left out on purpose. The one place a state machine would apply — a teacher account's ACTIVE/DISABLED/BANNED lifecycle, or a mark-edit-request's PENDING/APPROVED/REJECTED status — is a two- or three-state enum with no intermediate transitions or side-effect-heavy states worth drawing out. A one-line description in the relevant FR (2.3.21, 2.3.8) covers it better than a diagram would.

3.3 Architectural Design

The system is four layers, and every one of the three apps goes through all four the same way — there's no shortcut where cmsadmin talks to Postgres directly while cmsteacher goes through a different path. Compose screens hold a ViewModel each; the ViewModel calls into cmscommon's Repository layer, which is the only code that knows about DTOs, Supabase table names, or SQL-shaped queries. Below that, a Repository either reads/writes Room (for the handful of high-frequency, cached features) or talks straight to the Supabase Kotlin SDK, which is what actually reaches Postgrest, GoTrue, Storage, or an Edge Function over the network.

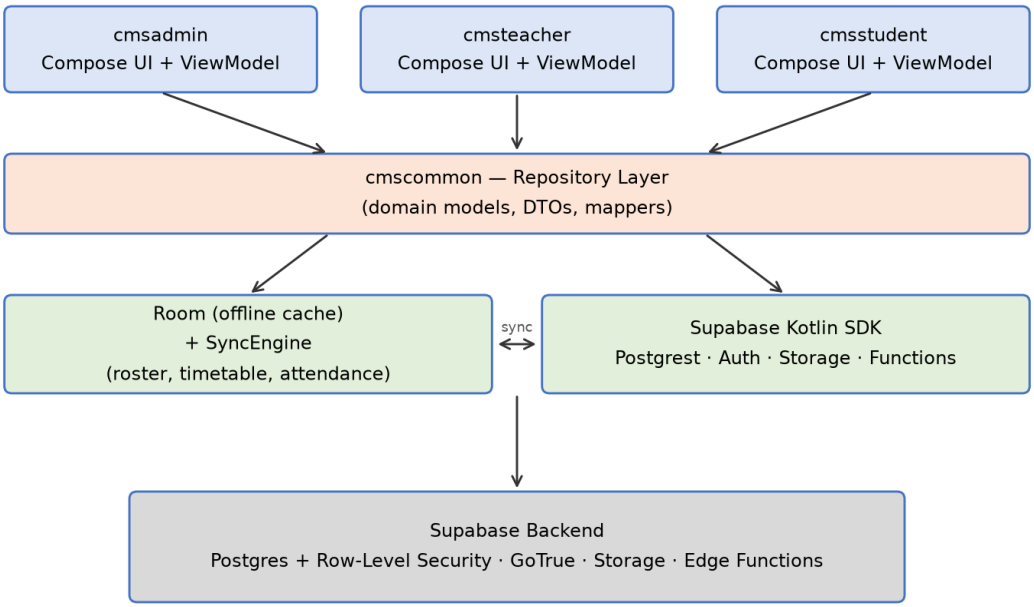

Figure 3-1 CMS layered architecture

This is closest to a layered/client-server hybrid rather than a textbook MVC or MVVM-only pattern — MVVM describes the top two layers (Compose View + ViewModel), but the Repository/Room/Supabase split underneath is really about where data lives and how fresh it needs to be, which MVVM alone doesn't prescribe. The actual enforcement of who can do what isn't in this diagram at all: it lives in Postgres RLS policies at the very bottom, which is why the same architecture serves three different roles without three different code paths.

3.4 Data Design

The domain here maps almost one-to-one onto Postgres tables — a department, a session, a student, a mark, a fee head are each one row somewhere, and the interesting structure is in the foreign keys and RLS policies between them, not in an object hierarchy. The data dictionary below is grouped by the same five domains used everywhere else in this report (identity, academic structure, financial, notices, analytics), pulled directly from the live schema rather than reconstructed from memory, so it reflects exactly what's deployed.

A few conventions run through the whole schema. Every foreign-key-style reference uses the natural key, not a surrogate integer — a session's ID is literally {deptId}_{startYear}_{shift} (e.g. it_2021_MORNING), and a student's ID within a session is {sessionId}_{rollNumber} — because those keys are genuinely stable and human-readable, and it means an admin looking directly at the database can identify a row without a join. Timestamps are timestamptz throughout, defaulting to now() at the database level rather than being set from the client, so a clock skew on someone's phone can't misdate a record.

3.4.1 Data Dictionary

Identity

Who a signed-in account is, and which role/roster row it resolves to. session_students carries a rich profile (30+ columns covering contact details, guardians, emergency contacts, and enrollment status) beyond what's listed here — only the fields that other tables actually reference are shown.

Entity.Field	Type	Description
profiles.id	uuid (PK)	Matches the GoTrue auth user ID — the JWT subject.
profiles.email	text	Normalized identity key used across the schema.

profiles.role	enum	ADMIN / TEACHER / STUDENT.
profiles.linked_session_id / linked_roll	text	Set once a student account is approved (FR-20); together they point at one session_students row.
profiles.status	enum	Mirrors the account's lifecycle status (see teachers.status).
teachers.email	text (PK)	Normalized identity key; also the FK target from timetable_periods.teacher_email.
teachers.status	enum	ACTIVE / DISABLED / BANNED (FR-21).
teachers.can_approve_link_requests / can_edit_timetable / can_send_notifications / can_manage_datesheets	boolean	Per-teacher permission flags, admin-set (FR-21).
session_students.session_id + roll_number	text (composite PK)	Natural key — no surrogate ID.
session_students.linked_email	text	Set by FR-20's approval; empty string until linked.
session_students.gpa / cgpa	numeric	Snapshot updated by the record_semester_result RPC (FR-9), not written directly.
session_students.enrollment_status	enum	ACTIVE / WITHDRAWN / etc.

Academic Structure

The core of the schema — departments own sessions, sessions own everything else (subjects, timetable, attendance, marks, results).

Entity.Field	Type	Description
departments.dept_id	text (PK)	No underscores allowed — session IDs split on '_' to recover this.
academic_sessions.session_id	text (PK)	{deptId}_{startYear}_{shift}, e.g. it_2021_MORNING.
academic_sessions.current_semester	int	The one field FR-2's "promote" action changes; drives which curriculum/timetable rows apply.
session_subjects (session_id, semester, course_code)	composite PK	Curriculum — replaced wholesale on save (FR-3), never diffed.
timetable_periods.id	uuid (PK)	day + start_time + end_time + primary_session_id are checked by a trigger to prevent double-booking a teacher/room (FR-4).
session_attendance (session_id, semester, course_code, date, roll_number)	composite PK	One row per student per class per day; is_late defaults false but is explicitly always-encoded (see 3.7) since it hit the DTO-default bug.
session_marks (session_id, semester, course_code, exam_type, roll_number)	composite PK	score is nullable until entered; UPDATE is admin-only (FR-6/FR-7).
mark_edit_requests.id	uuid (PK)	status: PENDING / APPROVED / REJECTED (FR-7/FR-8).
student_semester_gpa	composite PK	supply_courses is a text[]; written only via the

(session_id, roll_number, semester)		record_semester_result RPC, never a direct insert (FR-9).

Financial

Fees moved from per-department to per-session partway through the project (3.7 explains why); fines were always per-student.

Entity.Field	Type	Description
session_fees.session_id	text (PK)	One row per session — cadence is ANNUAL or SEMESTER, not per-fee-head.
session_fees.due_date / late_fine_note / payment_note	date / text / text	payment_note defaults to a fixed reminder that fees are payable at the accounts office, not through the app.
session_fee_heads (session_id, label)	composite PK	The actual line items (Tuition, Lab, Library, …); replaced wholesale on save, same delete-then-insert pattern as curriculum.
fines.id	uuid (PK)	category is an enum (LIBRARY / ATTENDANCE / EXAM / DISCIPLINARY / OTHER); amount and reason are required.

Notices

The broadcast/publication side of the system — calendar, exam datesheets, documents, exam papers, and admin notifications. Most rows here have a published/draft flag or an audience/target column, since visibility control is the whole point of this group.

Entity.Field	Type	Description
calendar_events.audience	enum	ADMIN / TEACHER / STUDENT / ALL (FR-14).
datesheets.published	boolean	Non-managing roles only see published=true rows (FR-15/FR-16).
datesheet_slots.datesheet_id	uuid (FK)	One datesheet has many slots — exam_date, times, room, invigilator.
documents.storage_path / body	text / text	Either or both may be set — an uploaded file, typed text, or both (FR-17).
documents.kind	enum	PROSPECTUS / RULES / REPORT / OTHER.
documents.search	tsvector	Full-text search column, generated — not written directly by the app.
exam_paper_submissions.storage_path	text	Points into the exam-papers Storage bucket; purged along with its row when a session's semester completes, to stay within the free-tier quota.
notifications.target_role / target_dept_id / target_session_id	enum / text / text	All nullable — a notification can be broadcast (all null) or scoped to any combination.

Analytics Views

None of these are tables — all five are Postgres views marked security_invoker, which is the one property that makes FR-22's tiered access work: querying a security_invoker view re-checks the caller's own RLS against the underlying tables, rather than running with whatever privileges created the view.

View.Field	Type	Description
session_overview	view	session_id, dept_id, shift, current_semester, students,

		avg_cgpa, avg_attendance — one row per session, joined from academic_sessions/session_students/session_attendance_summary.
at_risk_students	view	session_id, roll_number, name, cgpa, attendance — flags cgpa < 2.00 or attendance < 75%.
exam_stats	view	session_id, semester, course_code, exam_type, entered, avg_score, min_score, max_score, stddev, out_of, pass_rate — aggregated straight from session_marks.
student_gpa_progression	view	session_id, roll_number, semester, gpa, cgpa, term_label, result_status, class_position, gpa_delta (gpa_delta computed vs. the prior semester).
session_attendance_summary	view	session_id, semester, course_code, roll_number, present, absent, leave, total_marked, percentage — the aggregate every other attendance-percentage figure in the app is built from.

3.5 User Interface Design

From the user's side, every screen in all three apps follows the same shape: a header (eyebrow label, title, a short accent rule), then either a form, a list of LedgerCards, or a HubNavCard grid, and — where relevant — a floating action button for the one "create new" action that screen supports. A destructive action (deleting a session, banning a teacher, rejecting a request) always routes through a confirmation dialog that names what else is affected, never a bare "Are you sure?".

3.5.1 Screen Images

[Screenshots skipped for this draft — no Android emulator/device was available in the environment this report was generated in. Before submission, capture and insert screenshots of: Admin (Departments, Session Detail, Mark Edit Requests queue), Teacher (Marks Entry showing a locked score + edit-pencil, Semester Results), and Student (Home, Results, Fee Challan) — roughly 6-8 screens covering one from each major module.]

3.5.2 Screen Objects and Actions

Marks Entry (teacher): each roster row shows ROLL · STUDENT NAME · SCORE. For a not-yet-saved student, SCORE is an editable numeric field with live validation (a red caption appears the instant the value is negative or exceeds the exam's maximum). Once saved, SCORE becomes a static label plus an edit-pencil icon button; tapping it opens the Request Mark Edit dialog (current score, a new-score field with the same live validation, an optional reason, Send Request). A "Pending review" caption replaces the pencil while a request is outstanding, and the pencil is disabled so a second request can't be filed on top of the first.

Mark Edit Requests (admin): each queue card shows the course/exam/roll, current score → requested score, the teacher's reason, and two actions — Approve and Reject — with no intermediate confirmation step, since the action itself is already the confirmation (the request disappears from the queue once acted on).

3.6 Behavioural Model

Four flows are walked through as sequence diagrams below — chosen because each one crosses at least three of the four architectural layers (3.3) and each one has a non-obvious step that a class diagram alone wouldn't surface: login resolves a role before the app can show anything; the marks flow is the whole point of the mark-lock design (2.3.6-2.3.8); attendance marking is the highest-frequency write in the system; and document publish/download is the one flow that touches Storage rather than just Postgrest.

3.6.1 Login Flow

The non-obvious step here is RoleResolver: a JWT alone doesn't tell the app whether it's talking to an admin, teacher, or student — that still has to be looked up from the profiles/teachers rows, and the resolver checks the local Room cache before it checks the network so a previously-signed-in user doesn't stall on a slow connection just to find out their own role again.

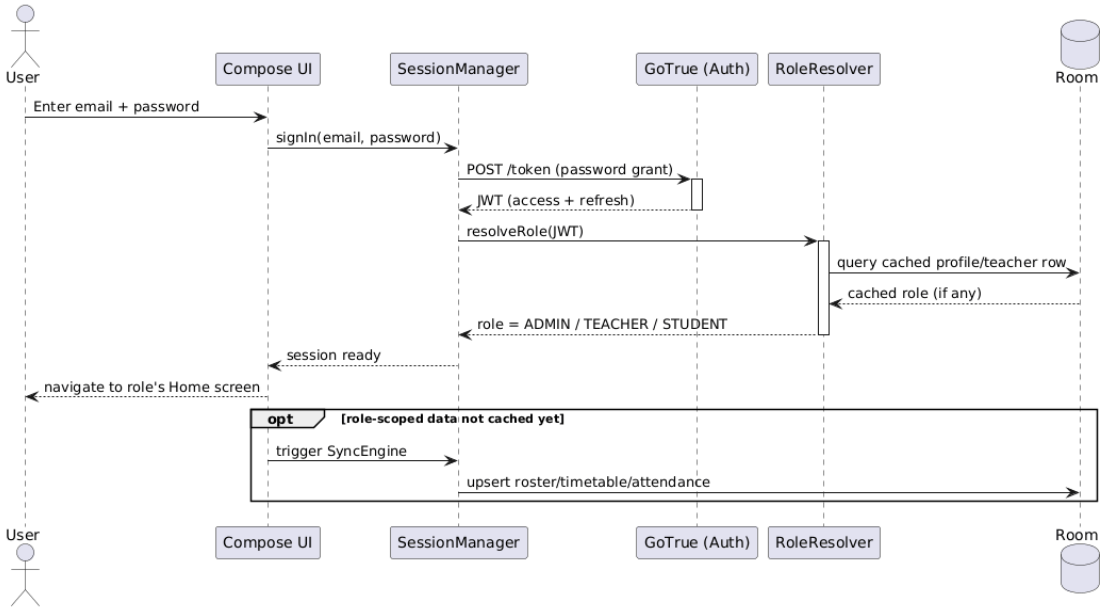

Figure 3-2 Login sequence

3.6.2 Marks Entry, Edit Request, and Approval

This is the one flow in the whole system built entirely around a single business rule (2.3.6-2.3.8): once session_marks.score exists for a student, only an admin-gated UPDATE can change it, so a teacher's own correction has to become a request row instead of a second write to the same table. The three-part split below — first entry, later correction, admin review — mirrors the three FRs it implements one for one.

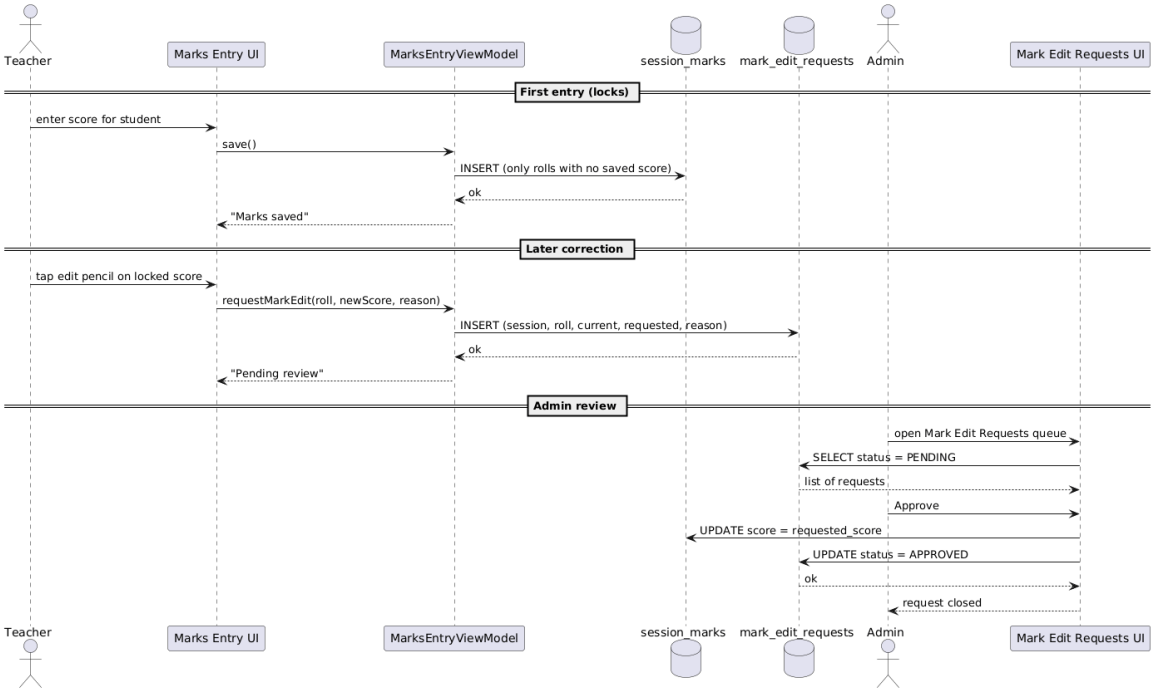

Figure 3-3 Marks entry / edit-request / approval sequence

3.6.3 Attendance Marking

The roster load reads from Room, not the network — this is the highest-frequency screen in the system (2.4.1), so it has to open instantly even on a weak connection. The save is the opposite: it goes straight to Postgrest as one bulk insert, and only updates the local cache afterward, so the source of truth for a write is always the server, never the offline copy.

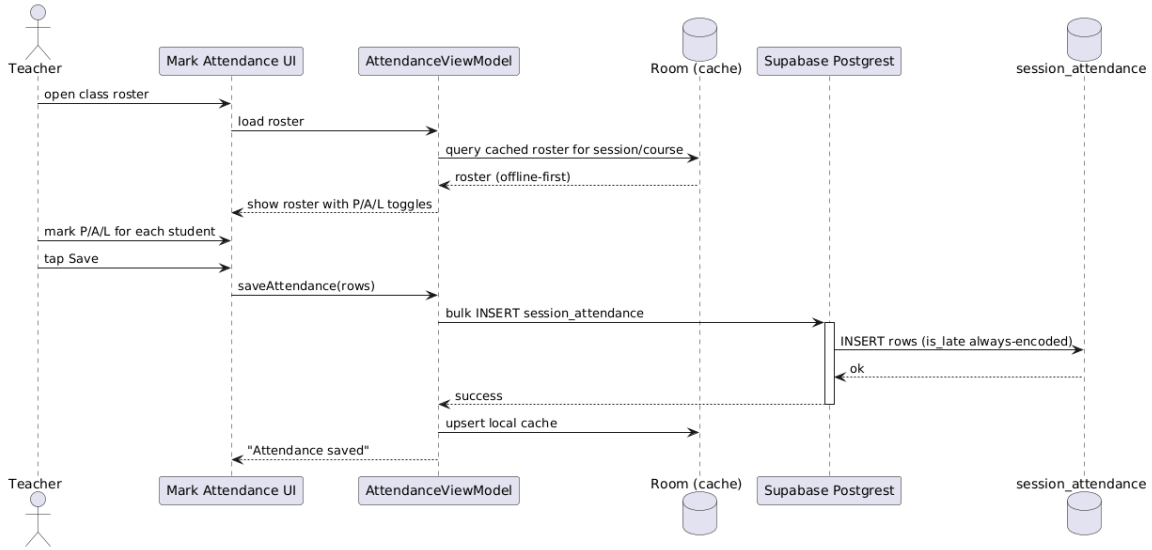

Figure 3-4 Attendance marking sequence

3.6.4 Document Publish and Download

This is the only one of the four flows that touches Supabase Storage rather than just Postgrest. The upload and the metadata insert are two separate calls, not one transaction — if the metadata insert failed after a successful upload, the blob would be orphaned in Storage with no documents row pointing at it, which is a real (if minor) gap noted again in 3.7.

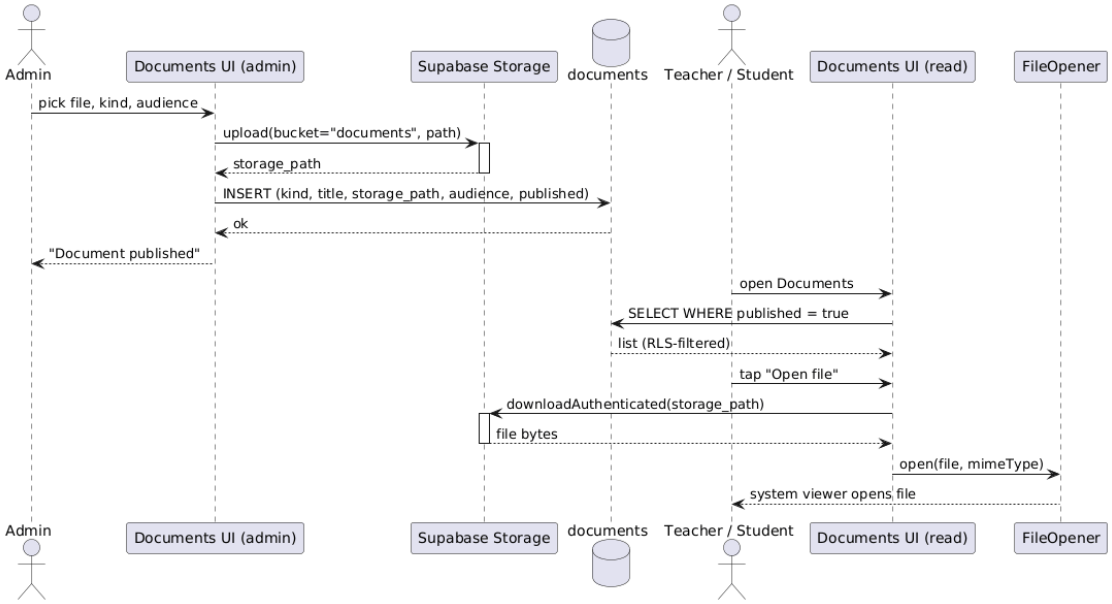

Figure 3-5 Document publish/download sequence

3.7 Design Decisions

- RLS over app-side checks. Every access rule (a teacher only sees their own sessions, a student only their own row) is a Postgres policy, not an if-statement in a Repository. The Insights feature is the proof this pays off: the same query serves three roles with zero role-branching in Kotlin. The cost is that RLS bugs are invisible in the Kotlin code — you have to go look at the policy — which is a real debugging cost the team accepted.
- Fees moved from per-department to per-session mid-project. The original design put a fee structure on each department, shared by every session under it. That broke the first time two intakes of the same department needed different tuition — a genuine requirements miss, not a technical one — so the schema, repository, and both the admin editor and student challan screen were rebuilt around session_fees/session_fee_heads instead.
- Delete-then-insert for small, infrequent collections. Curriculum, session fee heads, and timetable-adjacent lists are all replaced wholesale on save rather than diffed and patched. For a handful of rows edited a few times a semester, the simplicity of "delete the old set, insert the new one" beat writing and maintaining a diffing algorithm nobody would notice the absence of.
- Direct Postgrest over Room caching, for low-traffic features. Fees, fines, calendar, datesheets, documents, and insights all skip the offline cache entirely and read straight from the database. Only roster, timetable, and attendance — the screens opened dozens of times a day — got the Room + SyncEngine treatment. Caching everything would have meant maintaining nine more Room tables for data that's read a handful of times a week.
- The DTO-default-collision rule. kotlinx.serialization's encodeDefaults=false drops any field equal to its Kotlin default; combined with Postgrest's bulk-upsert behaviour (missing columns get filled with NULL, not the database's own default), this silently NULLed out a NOT NULL column three separate times before the pattern was understood. The fix that stuck: any DTO field backing a NOT NULL column with no database-level default gets a sentinel Kotlin default that can never be a real value (empty string, not the semantically normal default) — audited into every new DTO written after the bug was found, not just the three that broke.
- Marks lock after first entry, no exceptions for the entering teacher. This was a deliberate, not incidental, restriction — the alternative (let a teacher edit freely, log the change) was considered and rejected because a log nobody has to look at doesn't function as an audit trail. Requiring admin sign-off, even for the teacher's own typo, was the actual point.

3.8 Summary

This chapter covered how the requirements from Chapter 2 became an actual system: a four-layer architecture with Postgres RLS doing the access-control work rather than the app, a data model documented in full in the data dictionary, four sequence diagrams walking through the flows with the most moving parts, and a candid account of the design decisions that changed mid-project — the fee-structure rework and the DTO-default bug chief among them. Chapter 4 moves from design to the actual implementation: the algorithms, the third-party SDKs, and the state of the code repository itself.

Chapter 4

Implementation

4. Implementation

This chapter is about what's actually running, not the code itself — pseudocode for the pieces worth walking through, the third-party SDKs the apps depend on, and the state of the Git repository the three apps and the shared module live in.

4.1 Algorithm

Two pieces of logic are worth showing as pseudocode rather than just describing in prose: recording a semester result (the one place GPA/CGPA actually gets written), and the live score-validation check that runs on every keystroke in Marks Entry (2.3.6).

Algorithm 1 record_semester_result (Postgres RPC, SECURITY DEFINER)

record_semester_result(session, roll, semester, gpa, cgpa, term_label, result, class_position, remarks, supply[])

```
Input: session id, roll number, semester number, gpa, cgpa, optional term label/result/
       position/remarks, optional supply-subject list
Output: none (writes two tables)

1:  if NOT (caller is_admin() OR caller teaches(session)) then
2:      raise exception 'not allowed'
3:  upsert student_semester_gpa row keyed on (session, roll, semester)
4:      on conflict: overwrite gpa/cgpa/term_label/result/position/remarks/supply
5:  update session_students.gpa, .cgpa for (session, roll)
6:      WHERE no row exists in student_semester_gpa for this student with a HIGHER
7:            semester number than the one just recorded
8:  -- guarantees the snapshot always reflects the LATEST completed semester,
9:  --   even if an earlier semester is (re-)recorded out of order
```

Algorithm 2 scoreError(raw, maxMarks) — live validation in Marks Entry

scoreError(raw: String, maxMarks: Int): String?

```
Input: raw text currently in the score field, the exam type's max marks
Output: an error message to show, or null if the field is valid so far

1:  if raw is blank then return null        -- empty is not yet an error
2:  n <- raw.trim().toIntOrNull()
3:  if n is null then return "Numbers only"
4:  if n < 0 then return "Can't be negative"
5:  if n > maxMarks then return "Max is {maxMarks}"
6:  return null
```

4.2 External APIs/SDKs

Everything below is a real dependency the apps build against today, not an aspirational list — versions match 1.9's Tools and Technologies table.

API / SDK	Purpose	Used in
Supabase Postgrest-kt	Typed REST client over the Postgres schema — every Repository's read/write path.	cmscommon/data/repository/*.kt
Supabase Auth-kt (GoTrue)	Sign-in, session/token refresh, password reset.	SessionManager, RoleResolver
Supabase Storage-kt	Upload/download for exam papers and documents.	ExamPaperSubmissionRepositoryImpl,

		DocumentRepositoryImpl
Supabase Functions-kt	Invokes Edge Functions for service-role-only actions (create teacher account, change teacher status).	AdminUserProvisioner
Ktor (OkHttp engine)	The HTTP client every Supabase-kt module runs on.	Configured once in SupabaseModule (Hilt)
kotlinx.serialization	JSON (de)serialization for every DTO.	All *Dto.kt classes
Room + Room-ktx	Local offline cache + DAOs for roster/timetable/attendance.	cmscommon/data/local/*
Hilt	Dependency injection across all three apps and the shared module.	*Module.kt, @HiltViewModel classes
Jetpack Compose + Material3	All UI in all three apps.	Every feature/*/*.kt screen

4.3 Code Repository

Git is the version-control tool for the whole project — all three apps, the shared cmscommon module, and the Supabase migrations/Edge Functions live in one repository.

Git Repository Link: https://github.com/hadiyasaleem/CMS

4.3.1 Metrics of the Git Repository

These reflect the state of the repository at the time of writing (git log/git shortlog on the default branch), and will keep changing as work continues.

Metric	Value
Commits	107
Branches (local + remote)	9
Contributors	Hadia, Sharfa Kiran, Syeda Laraib Qamar Kazmi (some appear under more than one git identity/machine)
Latest merged PR	#20 — Attendance fragment and activity

Note on the contributor list: several team members show up under two Git identities (e.g. "sharfakiran" and "Sharfa Kiran", "hadiyasaleem" and "Hadiya Saleem") because Git config wasn't consistent across every machine used during development — the commit counts above are real, but attributing them cleanly per person would need a .mailmap rather than a straight git shortlog.

4.4 Summary

This chapter grounded the design in what's actually implemented: the two algorithms worth walking through in pseudocode, the real external SDKs each app depends on, and the state of the Git repository itself. Chapter 5 turns to testing — what's been verified, what's only been compiled, and what's still pending an actual device run.